

Hierarchical Caching

Deep Dive into SGLang — Chapter 9

Yin Yang

Foundation Books Project

Where this chapter sits

This is the chapter that keeps prefix reuse valid *after* KV pages leave GPU memory.

- Chapter 8 proved two requests share a prefix and protected its pages *on the device*.
- Device KV memory is too small to keep every reusable prefix resident.
- HiCache spreads the same prefix across **device, host, and storage** tiers — without breaking the reuse contract.

Goal: separate *preserved* state (the bytes survive) from *executable* state (attention can read it now). Load-back is the bridge.

The gap: preserved vs. executable

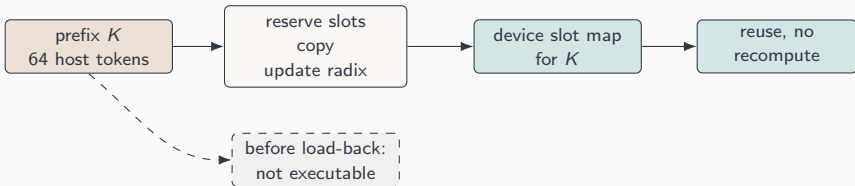
The lifecycle of one cached prefix

Correctness, scope, and evidence

**The gap: preserved vs.
executable**

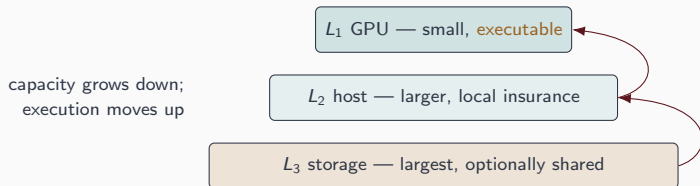
A host hit is not yet an executable hit

The prefix is found — but it sits in host DRAM while attention needs device-readable KV slots.



A radix match proves identity; it does not prove residency. HiCache adds the missing movement layer between the two.

The three tiers



- L₁ means “can run now.” L₂/L₃ mean “can avoid recompute *after* movement succeeds.”
- These are HiCache KV tiers, **not** CPU cache levels.

Load-back is promotion $L_i \rightarrow L_1$; demotion is its inverse under pressure.

The lifecycle of one cached prefix

Five recurring checks

Local match report the executable L_1 length and the longer L_1/L_2 length — by reading metadata, copying no bytes.

Load-back count a lower-tier hit only after slots are reserved, bytes copied, the transfer is acknowledged, and indices are published.

Demotion + write policy decide *when* lower tiers receive copies; protect nodes whose copy is not yet real.

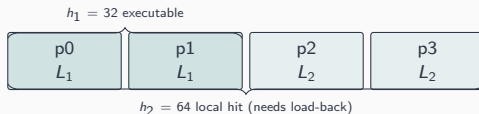
Storage prefetch a stop rule for an in-flight L_3 read; publish only completed, page-aligned pages.

Rank agreement every participating GPU agrees on the completed prefix length before shared cache state changes.

One invariant ties them together — the tier-alignment invariant.

The running example: the joke prefix K_{joke}

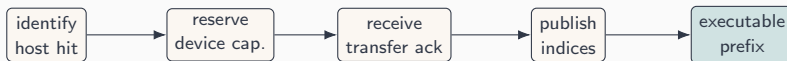
A 64-token shared chat prefix, page size $P = 16$, so $k = 4$ pages.



Local match returns $h_1 = 32$, $h_2 = 64$. The scheduler can run 32 tokens now, or ask the controller to load back pages 2–3 before counting them.

The readiness invariant, drawn

A lower-tier segment becomes **executable** only after *every* step:



$h_e \leq h$, executable only when device indices are *published*.

Leave out any term and you get a bug: a host hit without capacity cannot be placed; a reserved slot without completed data cannot be read.

From concept to code: a hierarchical match

The match returns device hits *and* how much more is host-backed.

```
from python/sglang/srt/mem_cache/hiradix_cache.py

host_hit_length = 0
last_host_node = last_node
while last_node.evicted:
    host_hit_length += len(last_node.host_value)
    last_node = last_node.parent
while not last_host_node.backuped:
    last_host_node = last_host_node.parent
return MatchResult(device_indices=value, last_device_node=last_node,
                    last_host_node=last_host_node,
                    host_hit_length=host_hit_length)
```

Two lengths, one walk: `device_indices` is the executable h_1 ; `host_hit_length` is the extra L_2 prefix that must be loaded back before it counts.

Correctness, scope, and evidence

One invariant rules them all

Tier-alignment invariant

A hierarchical-cache transition is valid only when the **prefix key**, **device indices**, **lower-tier copy**, **transfer ack**, **cache event**, and **rank state** all name the *same* reusable prefix segment.

It rules out three classic mistakes:

1. treating **host_hit_length** as if it were already in **device_indices**;
2. freeing host pages while an in-flight transfer still needs them;
3. emitting a cache event before the tier transition is real.

What HiCache owns — and what it does not

HiCache owns

- tier residency of a known prefix segment,
- write/load timing and protection,
- the four backend promises: exists, batch-get, batch-set, completion.

Neighbors own

- allocator: free lists, slot ownership,
- radix cache: the logical prefix-hit definition,
- backend: transport, durability, failure mode,
- scheduler: admission, P/D placement.

Backends (Mooncake, NIXL, file, 3FS) may change latency and capacity, but never prefix-key semantics or page order.

Honest measurement

- A reachable host or storage path is only a **compatibility check** — not a speedup.
- A hierarchy claim needs: a baseline, tier-hit attribution, transfer cost, *and* blocked-scheduler-step accounting.
- The reviewed Thor/Qwen artifact recorded device-prefix reuse but *not* host hits, load-back counters, or transfer latency.

So this edition reports no HiCache speedup. Hierarchical caching is a state-placement mechanism whose measurement burden is higher than ordinary radix reuse.

What to carry forward

1. Hierarchical caching separates **prefix identity** from **executable residency**.
2. **Load-back** is the only thing that turns a host/storage hit into a device hit — and only when acknowledged.
3. **Demotion** changes residency, not identity; **protection** guards transitions that are not yet safe.
4. **HiSparse** reapplies the same residency lesson to sparse decode reads.
5. Every mechanism has a **home in the SGLang source** — `hiradix_cache.py`, `cache_controller.py`, `hicache_storage.py`.

Next: Section 1 makes the gap between preserved and executable state precise, one tier at a time.